Calibración del Monte Carlo Permutation Test (MCPT) en el S&P 500

Validación empírica del test con tres modelos de verdad conocida: Oráculo, LightGBM (overfit) y Random

Reynaldo Lazcano Armenta, Juan Pablo Cordero Mayorga y Jerónimo Deli Larios —

2026-05-15

Table of contents

1 Introducción al problema **2**
1.1 Objetivo y pregunta de investigación . . . 3
1.2 Preguntas específicas . . . 3

2 Marco teórico **4**
2.1 El test de permutación de Monte Carlo (MCPT) . . . 4
2.2 ¿Qué permutar y por qué? (lo más importante) . . . 4
2.2.1 Permutación de precios (Masters) . . . 5
2.2.2 Permutación del target — *Y-permutation* (Masters) . . . 5
2.2.3 El problema oculto: la correlación serial en target y features . . . 5
2.3 Walk-forward vs in-sample . . . 6
2.4 ¿Qué es exactamente un *sliding walk-forward*? . . . 7
2.4.1 Esquema visual . . . 8
2.4.2 ¿Por qué se permuta exactamente así? . . . 9
2.5 Estadístico: Profit Factor . . . 10

3 Metodología y simulación **10**
3.1 Datos . . . 10
3.2 Features . . . 11
3.3 Los tres modelos . . . 12
3.4 Parámetros de simulación . . . 13

4 Mecánica de la permutación (la parte central) **13**
4.1 In-sample: permutar el par (y, r) con un único índice . . . 14
4.2 Walk-forward: permutar **una sola vez por permutación** sobre el rango OOS 15

5 Resultados e interpretación **18**
5.1 Tabla resumen . . . 18
5.2 In-Sample MCPT — histogramas . . . 19
5.2.1 Oráculo . . . 19
5.2.2 LightGBM (overfit) . . . 20
5.2.3 LightGBM clean (sólo OHLC crudo) . . . 21
5.2.4 Random puro . . . 22
5.3 In-Sample MCPT — curvas de equity . . . 22
5.3.1 Oráculo . . . 23
5.3.2 LightGBM (overfit) . . . 23
5.3.3 LightGBM clean . . . 24
5.3.4 Random . . . 25
5.4 Walk-Forward MCPT — histogramas . . . 25
5.4.1 Oráculo . . . 25
5.4.2 LightGBM (overfit) — el resultado clave del reporte . . . 26
5.4.3 Random . . . 27
5.5 Walk-Forward MCPT — curvas de equity . . . 27
5.5.1 Oráculo . . . 28
5.5.2 LightGBM . . . 29
5.5.3 Random . . . 30
5.6 Discusión del sesgo IS-LGBM . . . 30

6 Conclusiones **31**
6.1 Hallazgos principales . . . 31
6.2 Respuesta a las preguntas iniciales . . . 31
6.3 Limitaciones . . . 32
6.4 Trabajos futuros . . . 32

7 Referencias **32**

8 Anexo de código **32**
8.1 Configuración . . . 33
8.2 Descarga de datos . . . 33
8.3 Función de Profit Factor . . . 33
8.4 In-sample MCPT (completo) . . . 34
8.5 Walk-Forward MCPT (completo) . . . 35
8.6 Notebook completo . . . 36

1 Introducción al problema

En finanzas cuantitativas hay una pregunta que aparece una y otra vez: cuando un modelo de *machine learning* muestra una curva de equity hermosa en un backtest, **¿realmente encontró**

una señal predictiva en el mercado, o simplemente memorizó ruido? Esta pregunta no es retórica: el sobreajuste (*overfitting*) es la causa principal por la que sistemas de trading que parecen rentables en papel pierden dinero cuando se les pone capital real.

El problema fundamental es que con suficientes parámetros y suficientes pruebas, **cualquier modelo puede aprender los accidentes históricos del precio** y exhibir métricas espectaculares en la muestra de entrenamiento. La pregunta correcta no es "¿mi sistema gana dinero en backtest?" sino "**¿mi sistema gana significativamente más de lo que ganaría un sistema sin señal sobre datos sin información predictiva?**" Esa es exactamente la pregunta que responde el **Monte Carlo Permutation Test (MCPT)** descrito por Timothy Masters en *Testing and Tuning Market Trading Systems*.

1.1 Objetivo y pregunta de investigación

Este reporte **no** intenta construir el mejor sistema de trading sobre S&P 500. Es un trabajo de **calibración metodológica**. Queremos responder, empíricamente, una sola pregunta:

> **¿El MCPT realmente discrimina entre modelos con señal genuina y modelos que sólo memorizan ruido, cuando los aplicamos a un activo real (SPY)?**

Para responderla, diseñamos un experimento con tres modelos **donde nosotros conocemos la verdad**:

1. **Oráculo con ruido**: ve el target real y le suma ruido gaussiano. Por construcción tiene señal.
2. **LightGBM overfit**: gradient boosting con muchas features técnicas. Por construcción **debe** overfittear in-sample y fallar walk-forward.
3. **Random puro**: tira una moneda. Por construcción **no** tiene señal.

Si el MCPT funciona, debería darnos p-values bajos para el oráculo (señal real), p-values muy bajos in-sample pero altos walk-forward para el LGBM (overfitting clásico), y p-values aproximadamente uniformes para el random (control negativo).

1.2 Preguntas específicas

1. ¿El MCPT in-sample detecta sobreajuste? (LGBM: p-IS bajo, p-WF alto)
2. ¿El MCPT walk-forward detecta sistemas genuinos? (Oráculo: p-WF bajo)
3. ¿Bajo la hipótesis nula, el p-value es uniforme? (Random: p U[0,1])
4. ¿Cómo se contamina el test cuando las features están correlacionadas con la magnitud del retorno? (caso LGBM vs LGBM clean)

2 Marco teórico

2.1 El test de permutación de Monte Carlo (MCPT)

El MCPT (Masters, capítulo 7) es un test no paramétrico de hipótesis. Su lógica es la siguiente:

> **Hipótesis nula del MCPT**
>
> H_0: el modelo es **inútil**, es decir, no hay relación entre las features X y el target y. Bajo esta hipótesis, cualquier desempeño positivo observado se debe al azar.

La idea operativa es:

1. **Computar el desempeño real** del sistema con los datos en su orden original. Llamemos a este valor θ_{real}.
2. **Permutar aleatoriamente** los datos de manera que **destruyamos cualquier relación predictiva** entre X y y, pero **preservemos** las distribuciones marginales y las dependencias intra-X.
3. **Volver a correr** el sistema sobre los datos permutados y medir $\theta^{(j)}$.
4. **Repetir** m veces (m del orden de cientos o miles).
5. **Comparar** θ_{real} contra la distribución $\{\theta^{(1)}, \dots, \theta^{(m)}\}$.

El p-value se calcula como:

$$p\text{-value} = \frac{k+1}{m+1}, \quad k = \#\{j : \theta^{(j)} \geq \theta_{\text{real}}\}$$

El "+1" en numerador y denominador es la corrección de Masters (la corrida original cuenta como una observación válida bajo H_0). Si θ_{real} cae a la derecha de casi todas las permutaciones, k es chico y el p-value es chico: hay evidencia contra H_0, es decir, evidencia de señal genuina.

2.2 ¿Qué permutar y por qué? (lo más importante)

Esta es la decisión crítica de todo el experimento, y donde Masters dedica buena parte del capítulo. Existen dos grandes estrategias:

2.2.1 Permutación de precios (Masters)

Apropiada para **sistemas de trading puros** donde el sistema mira la historia de precios y toma decisiones de long/short. Se deconstruye el precio en cambios (log-returns), se barajan los cambios, y se reconstruye una historia sintética manteniendo el precio inicial y final (preservando el trend). El sistema se re-entrena sobre el "mercado sintético".

2.2.2 Permutación del target — *Y-permutation* (Masters)

Apropiada para **modelos predictivos** donde tenemos pares (X_t, y_t) y queremos romper el link entre features y target sin tocar la estructura interna de X. Es exactamente nuestro caso: tenemos features técnicas sobre SPY y queremos predecir el signo del retorno del día siguiente.

Nosotros usamos Y-permutation. La justificación de Masters es:

> "Indicator sets must not permute with respect to one another, only with respect to the target. This preserves intraset correlation, which is critical to correct testing." (Masters)

Es decir: si tenemos como features la media móvil de 10 días, la de 20, la de 50 y la volatilidad realizada, esas features están **correlacionadas entre sí en el tiempo real**. Si las permutamos de manera independiente generaríamos combinaciones imposibles en el mercado real (volatilidad alta con todas las medias móviles planas, por ejemplo). El MCPT exige que las permutaciones tengan **la misma probabilidad de ocurrir bajo** H_0 que en la realidad. Permutar features independientemente viola esto.

La solución limpia: dejamos X intacto y barajamos sólo y (junto con los retornos r que se usan para evaluar PnL). Así rompemos el emparejamiento $X_t \to y_t$ sin tocar la estructura interna de X.

2.2.3 El problema oculto: la correlación serial en target y features

Masters advierte de un segundo problema:

> "Serial correlation in just one or more predictor variables, or in just the target, is harmless. [...] But if both are serially correlated, permutation will destroy this property, and we will be in the situation of processing pairings that could not occur in real life."

Las features técnicas (medias móviles, volatilidades realizadas) tienen **enorme autocorrelación serial**: el valor en t y en $t-1$ son casi iguales. Si el target o el retorno también tienen autocorrelación, el MCPT se rompe.

Nuestro target es el signo del log-return, que tiene autocorrelación cercana a cero en daily data (consistente con la hipótesis de eficiencia débil). Pero hay un problema más sutil: algunas de nuestras features están correlacionadas con la **magnitud** del retorno futuro (por ejemplo, la volatilidad realizada predice la magnitud del próximo movimiento, aunque no su signo). Esto puede inducir un sesgo en favor de la corrida real, porque el modelo no permutado se beneficia de saber **cuándo** los retornos son grandes.

Para cuantificar este sesgo entrenamos LightGBM en dos configuraciones:

- **LGBM (full)**: con todas las features técnicas (27 columnas).
- **LGBM clean**: sólo con `open, high, low, close` crudos (4 columnas), que tienen correlación mínima con $|r_{t+1}|$.

La diferencia entre los dos p-values aísla el sesgo inducido por features correlacionadas con la magnitud.

2.3 Walk-forward vs in-sample

Masters distingue tres usos del MCPT:

1. **Testing a fully specified system** sobre datos OOS.
2. **Testing the training process**, con énfasis en detectar overfitting (in-sample).
3. **Testing a model factory** (walk-forward).

Nosotros corremos los dos extremos:

- **MCPT in-sample**: entrenamos el modelo sobre todos los datos y medimos el Profit Factor (PF) in-sample. Bajo H_0, permutamos y y re-entrenamos. Este test detecta overfitting: un modelo overfit tendrá PF altísimo en la corrida real, pero también en las permutaciones (porque vuelve a memorizar ruido), así que el p-value será alto.

  > "An overfitted trading system will perform well not only on the original data but on permuted data as well. […] All in-sample performances, permuted and unpermuted, will be excellent, and the original performance will not stand out from its permuted competitors." (Masters)

- **MCPT walk-forward (WF)**: en cada fold, entrenamos con una ventana de los últimos 8 años y predecimos los próximos 3 años (re-entrenando cada 756 días). Concatenamos todas las predicciones OOS y medimos PF. Bajo H_0, permutamos y y los retornos en el rango OOS conjunto **una sola vez por permutación** y corremos todo el walk-forward de nuevo.

Aquí la regla de Masters es respetar el principio "**la primera ventana de training nunca aparece en OOS**": por lo tanto **no permutamos esa primera ventana**, sólo el rango OOS.

2.4 ¿Qué es exactamente un *sliding walk-forward*?

El walk-forward es la forma honesta de evaluar un modelo en series de tiempo. En *machine learning* tradicional uno divide los datos aleatoriamente en train/test, pero **eso no funciona en finanzas**: si entrenamos con datos de 2020 y testeamos con datos de 2010, estaríamos usando el futuro para predecir el pasado (*look-ahead bias*). Cualquier modelo se vería artificialmente bueno.

El walk-forward respeta el orden temporal. Existen dos variantes:

- **Anclado (*anchored*)**: la ventana de training **crece** con el tiempo. Siempre empieza en el primer dato y se extiende hasta el momento actual. El modelo "recuerda" toda la historia.
- **Deslizante (*sliding* o *rolling*)**: la ventana de training tiene **tamaño fijo** y se mueve hacia adelante. El modelo "olvida" datos viejos y siempre se entrena sobre las últimas N observaciones.

Nosotros usamos sliding walk-forward. Asumimos que el mercado tiene memoria limitada: los regímenes cambian, las volatilidades cambian, las correlaciones cambian. Entrenar sobre una ventana móvil hace que el modelo se adapte a las condiciones recientes en lugar de arrastrar régimenes obsoletos.

2.4.1 Esquema visual

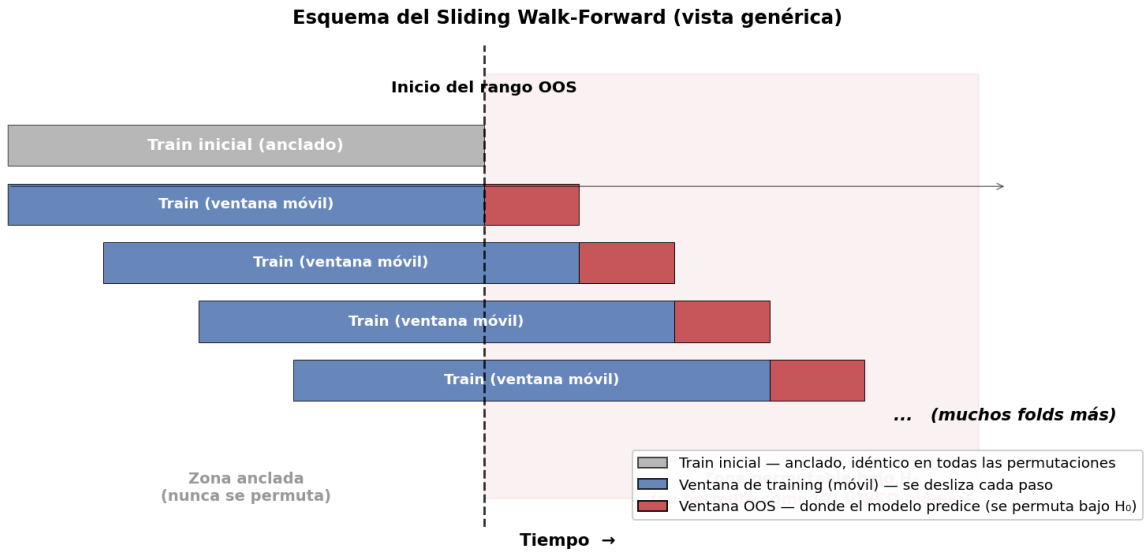

Figure 1: Esquema genérico del sliding walk-forward. La ventana de training (azul) tiene tamaño fijo y se desliza un paso a la vez; en cada paso, el modelo predice sobre la ventana OOS inmediatamente siguiente (rojo). La zona inicial (gris) queda anclada.

Cada **fold** consiste en:

1. Una **ventana de training** de tamaño fijo (azul) sobre las observaciones más recientes hasta ese momento.
2. Una **ventana OOS** inmediatamente posterior (rojo), donde el modelo predice **sin haber visto esos datos**.
3. Entre fold y fold, ambas ventanas se desplazan juntas hacia adelante.
4. Las predicciones de todas las ventanas OOS se **concatenan** en una sola serie OOS continua. Sobre esa serie calculamos el Profit Factor final.

En nuestro experimento sobre SPY, con ventana de training de 8 años y ventana OOS de 3 años sobre ~26 años de datos, terminamos con **exactamente 6 folds OOS** (5 folds completos de 3 años + un fold final de ~2.5 años, dado que el rango OOS total es ~17.5 años). Cada fold re-entrena el modelo desde cero sobre su ventana — no hay *online learning* ni transferencia entre folds. Folds OOS largos (3 años) en lugar de cortos (3 meses) hacen el test más exigente: el modelo tiene que generalizar lejos de su training set, y refleja mejor la práctica realista de un sistema que se re-entrena cada cierto tiempo en lugar de cada trimestre.

Qué se permuta en walk-forward

Mirando el diagrama: **sólo se permuta la zona roja**, es decir, el rango OOS conjunto a partir del final del train inicial.

- **Zona gris (train inicial)**: anclada. No se toca. Es idéntica en la corrida real y en todas las permutaciones.
- **Zona roja (todo el OOS)**: bajo H_0 permutamos el par (y, r) con un único índice aleatorio sobre **todo** ese rango, de una vez. Después el walk-forward corre normalmente sobre el dataset resultante: las ventanas de training móviles van pasando por encima del rango permutado y re-entrenando con y barajado.

2.4.2 ¿Por qué se permuta exactamente así?

Hay dos detalles clave:

1. El train inicial queda anclado. En la corrida real, esos primeros datos sólo aparecen como *training* del primer fold; **nunca aparecen como ventana OOS**. Si los permutáramos, podrían acabar mezclados dentro de alguna ventana OOS de una corrida bajo H_0, inyectando observaciones (un crash, un período de alta volatilidad) que no estuvieron en la corrida real. Eso rompería la comparabilidad apples-to-apples del MCPT.

2. Todo el rango OOS se permuta como un único bloque. Una alternativa sería permutar cada fold por separado, pero eso destruiría la estructura temporal de largo plazo (tendencia, volatility clustering, autocorrelación) del rango OOS y trataría a cada fold como un universo aislado. Permutar el bloque completo preserva esa estructura.

En términos de código, esto se traduce en un solo `permutation` del par (y, r) sobre el rango OOS, seguido del walk-forward normal:

```
def _wf_one_perm(seed, model_name, X, y, log_ret):
    # 1) Copiamos y, ret. Pre-OOS queda igual (zona gris).
    y_used   = y.copy()
    ret_used = log_ret.copy()

    # 2) Permutamos el rango OOS conjunto (zona roja) con UN ÚNICO índice.
    #    y_used[t] y ret_used[t] se mantienen emparejados entre sí.
    rng = np.random.default_rng(seed)
    idx = rng.permutation(len(y) - TRAIN_WINDOW)
    y_used.iloc[TRAIN_WINDOW:]   = y.iloc[TRAIN_WINDOW:].values[idx]
    ret_used.iloc[TRAIN_WINDOW:] = log_ret.iloc[TRAIN_WINDOW:].values[idx]

    # 3) Walk-forward normal sobre el dataset semi-permutado.
```

```
    sig, ret = _wf_one(model_name, X, y, log_ret, y_used, ret_used)

    # 4) PF sólo sobre el rango OOS.
    sig_oos = sig.iloc[TRAIN_WINDOW:]
    ret_oos = ret.iloc[TRAIN_WINDOW:]
    return profit_factor(sig_oos, ret_oos)
```

Lo importante no es el modelo en sí, ni cuántos folds resultan — lo importante es **qué se permuta y qué no**. El train inicial queda intacto (zona gris), el rango OOS se baraja como un solo bloque (zona roja). Eso es lo que hace al test honesto.

2.5 Estadístico: Profit Factor

Para evaluar las señales usamos el **Profit Factor (PF)**:

$$\mathrm{PF} = \frac{\sum_{t:\mathrm{PnL}_t>0} \mathrm{PnL}_t}{\left|\sum_{t:\mathrm{PnL}_t<0} \mathrm{PnL}_t\right|}, \quad \mathrm{PnL}_t = \mathrm{signal}_t \cdot r_{t+1}$$

Es la razón entre dólares ganados y dólares perdidos. Por construcción $\mathrm{PF} > 1$ significa rentable; $\mathrm{PF} = 1$ es break-even; $\mathrm{PF} < 1$ es perdedor. Lo usamos porque es robusto a outliers y captura la calidad económica, no sólo la dirección.

3 Metodología y simulación

3.1 Datos

- **Activo**: SPY (proxy del S&P 500), descargado de Yahoo Finance.
- **Período**: 2000-01-03 a 2026-05-15.
- **Frecuencia**: diaria.
- **Total**: 6,632 barras OHLCV crudas; 6,432 filas tras construir features (descartamos lookbacks largos).
- **Target**: $y_t = \mathrm{sign}(r_{t+1})$, donde $r_{t+1} = \log(P_{t+1}) - \log(P_t)$. Reemplazamos los ceros por $+1$ (raro en daily data).
- **Balance**: 55% long, 45% short (consistente con el drift positivo del SPY).

3.2 Features

Construimos un set amplio (27 features) para que LightGBM tenga capacidad sobrada para overfittear:

```
def build_features(ohlcv: pd.DataFrame):
    df = pd.DataFrame(index=ohlcv.index)
    close, open_, high, low, vol = (
        ohlcv['Close'], ohlcv['Open'], ohlcv['High'], ohlcv['Low'], ohlcv['Volume']
    )
    log_close = np.log(close)
    log_ret   = log_close.diff()

    # Retornos a varios horizontes
    for n in [1, 2, 3, 5, 10, 21, 63]:
        df[f'ret_{n}d'] = log_close.diff(n)

    # Ratios sobre medias móviles
    for n in [10, 20, 50, 100, 200]:
        df[f'price_ma{n}_ratio'] = close / close.rolling(n).mean() - 1

    # Volatilidad realizada (anualizada)
    for n in [5, 10, 21, 63]:
        df[f'rvol_{n}d'] = log_ret.rolling(n).std() * np.sqrt(252)

    # Ratio de vol corta/larga, ATR normalizado
    df['vol_ratio']    = df['rvol_5d'] / (df['rvol_63d'] + 1e-9)
    tr = pd.concat([high - low,
                    (high - close.shift()).abs(),
                    (low  - close.shift()).abs()], axis=1).max(axis=1)
    df['atr_norm_14d'] = tr.rolling(14).mean() / close

    # Estructura intra-día y momentos de tercer/cuarto orden
    df['candle_body'] = (close - open_) / close
    df['gap']         = (open_ - close.shift()) / close.shift()
    df['hl_range']    = (high - low) / close
    df['rel_volume']  = vol / vol.rolling(20).mean()
    df['skew_21d']    = log_ret.rolling(21).skew()
    df['kurt_21d']    = log_ret.rolling(21).kurt()
    df['skew_63d']    = log_ret.rolling(63).skew()

    # Calendario
    df['day_of_week'] = ohlcv.index.dayofweek.astype(float)
```

```
    df['month']       = ohlcv.index.month.astype(float)

    target = np.sign(log_close.shift(-1) - log_close).replace(0, 1)
    df['_target'] = target
    df = df.dropna()
    return df.drop(columns=['_target']), df['_target']
```

Adicionalmente definimos `X_is` con sólo `open, high, low, close` crudos. Esta variante se usa para el experimento de **LGBM clean** que aísla el efecto de las features correlacionadas con la magnitud del retorno:

```
X_is = pd.DataFrame({
    'open':  spy['Open'], 'high':  spy['High'],
    'low':   spy['Low'],  'close': spy['Close'],
}, index=spy.index).reindex(X_full.index)
```

3.3 Los tres modelos

Cada modelo expone la misma interfaz: dado X (e idealmente y para entrenar), devuelve una `Series` de señales en $\{-1, +1\}$.

```
def predict_oracle(y_true, seed, noise_std=ORACLE_NOISE):
    """Modelo 1 - Oráculo: ve la y real, le suma ruido gaussiano, devuelve el signo.
    Con ORACLE_NOISE = 1.0 da señal moderada (PF   5)."""
    rng = np.random.default_rng(seed)
    noise = rng.normal(0, noise_std, size=len(y_true))
    return pd.Series(np.sign(y_true.values + noise),
                     index=y_true.index).replace(0, 1)

def train_lgbm(X, y):
    """Modelo 2 - LightGBM con capacidad suficiente para memorizar ruido."""
    y_bin = ((y.values + 1) / 2).astype(int)
    m = LGBMClassifier(num_leaves=31, min_child_samples=20,
                       learning_rate=0.05, n_estimators=200,
                       random_state=42, n_jobs=1, verbose=-1)
    m.fit(X.values, y_bin)
    return m

def predict_random(X, seed):
    """Modelo 3 - Random: moneda al aire. Control negativo."""
    rng = np.random.default_rng(seed)
```

```
    return pd.Series(rng.choice([-1.0, 1.0], size=len(X)), index=X.index)

def profit_factor(signal, log_ret):
    pnl = signal.values * log_ret.values
    pnl = pnl[~np.isnan(pnl)]
    wins = pnl[pnl > 0].sum()
    loss = np.abs(pnl[pnl < 0].sum())
    return wins / loss if loss > 1e-10 else float('inf')
```

3.4 Parámetros de simulación

Parámetro	Valor	Comentario
`N_PERM` (IS)	1,000	Permutaciones por modelo in-sample
`N_PERM` (WF)	1,000	Permutaciones por modelo walk-forward
`TRAIN_WINDOW`	2,016 días ($\approx$ 8 años)	Ventana de training para cada fold WF
`WF_STEP`	756 días ($\approx$ 3 años)	Tamaño de la ventana OOS / frecuencia de re-entrenamiento (resulta en 6 folds OOS)
`ORACLE_NOISE`	1.0	Std del ruido gaussiano en el oráculo
`RNG_SEED`	67	Semilla maestra (reproducibilidad)
`REAL_SEED`	7	Semilla de la corrida real (un draw fijo)
`N_JOBS`	8	Workers paralelos (joblib/loky)

Todas las semillas están fijadas. La reproducibilidad es exacta.

4 Mecánica de la permutación (la parte central)

Aquí dedicamos una sección entera a explicar **exactamente** cómo permutamos en cada test. Es la decisión metodológica más delicada y donde se gana o se pierde la validez del MCPT.

4.1 In-sample: permutar el par (y, r) con un único índice

En IS, el dataset es estático. Tenemos las matrices X (features), y (target ± 1) y r (log-retornos para evaluar PnL). Bajo H_0 queremos romper el link $X \to (y, r)$.

La regla de oro: y y r **deben permutarse con el mismo índice**. ¿Por qué? Porque $y_t = \text{sign}(r_{t+1})$. Si los permutamos por separado, romperíamos esa identidad y tendríamos targets imposibles dado los retornos. Mantener el par junto preserva la coherencia interna del "mundo permutado".

```
def _permute_pair(y, ret, seed):
    """Permuta y y ret con el MISMO índice aleatorio.
    Resultado: y_perm[t] y ret_perm[t] siguen emparejados entre sí,
    pero desalineados del X[t] original. Eso rompe X -> (y, ret)
    sin contaminar la coherencia interna del par."""
    rng      = np.random.default_rng(seed)
    idx      = rng.permutation(len(y))
    y_perm   = pd.Series(y.values[idx],   index=y.index)
    ret_perm = pd.Series(ret.values[idx], index=ret.index)
    return y_perm, ret_perm
```

Para cada uno de los tres modelos, una permutación significa cosas distintas:

Modelo	Qué hace bajo permutación
Oráculo	Usa la y **real** (no permutada) para generar su señal. Pero la señal se evalúa contra `ret_perm`. Resultado esperado: PF colapsa a ≈ 1 porque la señal "correcta" se aplica a retornos aleatoriamente reordenados.
LGBM	Re-entrena sobre (X, y_{perm}) y predice sobre X. La señal se evalúa contra `ret_perm`. Si LGBM es lo bastante flexible, **igual va a memorizar la y permutada** y producir un PF alto. Eso es exactamente lo que queremos detectar como overfitting.
Random	Ignora todo. Señal aleatoria evaluada contra `ret_perm`. Distribución bajo H_0 centrada en PF $= 1$.

```
def run_is_mcpt(model_name, X, y, log_ret, n_perm, n_jobs, real_seed):
    def _signal(y_for_training, seed):
```

```
        if model_name == 'oracle':
            return predict_oracle(y, seed=seed)              # oráculo SIEMPRE ve y real
        elif model_name in ('lgbm', 'lgbm_clean'):
            return predict_lgbm(train_lgbm(X, y_for_training), X)
        elif model_name == 'random':
            return predict_random(X, seed=seed)

    def _one_perm(seed):
        y_perm, ret_perm = _permute_pair(y, log_ret, seed)
        sig   = _signal(y_perm, seed)
        pf    = profit_factor(sig, ret_perm)
        return pf

    # Corrida real: nada permutado
    sig_real = _signal(y, real_seed)
    real_pf  = profit_factor(sig_real, log_ret)

    # m permutaciones en paralelo
    perm_pfs = Parallel(n_jobs=n_jobs, backend='loky')(
        delayed(_one_perm)(s) for s in range(1, n_perm + 1)
    )
    perm_pfs = np.array([pf for pf in perm_pfs if np.isfinite(pf)])

    k     = int((perm_pfs >= real_pf).sum())
    p_val = (k + 1) / (len(perm_pfs) + 1)        # fórmula de Masters
    return {'real_pf': real_pf, 'perm_pfs': perm_pfs, 'p_value': p_val}
```

4.2 Walk-forward: permutar una sola vez por permutación sobre el rango OOS

Aquí la cosa se pone más sutil. Masters es explícito: en walk-forward, **la primera ventana de training nunca aparece en OOS en la corrida original**, por lo tanto **no debe permutarse hacia adentro del rango OOS de las permutaciones**, porque podría inyectar datos atípicos (un crash, un período de alta volatilidad) que distorsionarían la comparación.

Nuestra estrategia (siguiendo la opción de Masters, "do a single permutation of all market changes after the first training fold"):

1. **Pre-OOS** (primeras 2,016 barras, 2000-01-03 a 2008-01-04 aprox.): **nunca se permuta**. Es el train inicial. La curva de equity es idéntica entre todas las permutaciones en esa zona.
2. **OOS conjunto** (todo lo que viene después): se permuta **una sola vez** por permutación, con un único índice aleatorio, manteniendo y y r emparejados.

3. Sobre el dataset resultante, corremos el walk-forward normalmente: ventana móvil de 8 años, re-entrenando cada 3 años (6 folds en total).

```
def _wf_one_perm(seed, model_name, X, y, log_ret):
    # 1) Copiamos y, ret. Pre-OOS queda igual.
    y_used   = y.copy()
    ret_used = log_ret.copy()

    # 2) Permutamos el rango OOS conjunto con un ÚNICO índice
    rng   = np.random.default_rng(seed)
    idx   = rng.permutation(len(y) - TRAIN_WINDOW)
    y_oos = y.iloc[TRAIN_WINDOW:].values[idx]
    r_oos = log_ret.iloc[TRAIN_WINDOW:].values[idx]
    y_used.iloc[TRAIN_WINDOW:]   = y_oos
    ret_used.iloc[TRAIN_WINDOW:] = r_oos

    # 3) Walk-forward normal sobre el dataset semi-permutado
    sig, ret = _wf_one(model_name, X, y, log_ret, y_used, ret_used)

    # PF sólo sobre el OOS
    sig_oos = sig.iloc[TRAIN_WINDOW:]
    ret_oos = ret.iloc[TRAIN_WINDOW:]
    valid   = ret_oos.notna()
    pf      = profit_factor(sig_oos[valid], ret_oos[valid])
    return pf
```

Dentro de cada fold walk-forward, el modelo se entrena con la `y_used` (potencialmente permutada) y se evalúa contra `ret_used` (consistentemente permutado). Esto rompe el link $X \to (y, r)$ **sólo dentro del rango OOS**. La parte pre-OOS sirve como "anclaje" común para todas las trayectorias.

```
def _wf_one(model_name, X, y, log_ret, y_used, ret_used, real_seed=REAL_SEED):
    n        = len(X)
    signals  = []; rets_eval = []

    # Train inicial (común a todas las permutaciones)
    X_init       = X.iloc[:TRAIN_WINDOW]
    y_init_real  = y.iloc[:TRAIN_WINDOW]
    y_init_train = y_used.iloc[:TRAIN_WINDOW]
    ret_init     = ret_used.iloc[:TRAIN_WINDOW]
    sig_init     = _wf_signal_for_window(
        model_name, X_tr=X_init, y_tr_for_training=y_init_train,
```

```
        y_real_window=y_init_real, X_win=X_init, seed=real_seed,
    )
    signals.append(sig_init); rets_eval.append(ret_init)

    # Folds OOS: 6 folds, cada uno re-entrena con su ventana de los 8 años previos
    for start in range(TRAIN_WINDOW, n, WF_STEP):
        end       = min(start + WF_STEP, n)
        X_tr      = X.iloc[start - TRAIN_WINDOW : start]
        y_tr      = y_used.iloc[start - TRAIN_WINDOW : start]
        X_te      = X.iloc[start : end]
        y_te_real = y.iloc[start : end]
        r_te      = ret_used.iloc[start : end]

        sig = _wf_signal_for_window(
            model_name, X_tr=X_tr, y_tr_for_training=y_tr,
            y_real_window=y_te_real, X_win=X_te,
            seed=real_seed + start,
        )
        signals.append(sig); rets_eval.append(r_te)

    return pd.concat(signals), pd.concat(rets_eval)
```

¿Por qué una sola permutación global y no una por fold?

Masters discute las alternativas: permutar todo el OOS como un único bloque, o permutar cada fold por separado. Elegimos la primera porque preserva mejor las propiedades estadísticas de largo plazo (tendencia, vol, autocorrelación) del rango OOS, en lugar de tratar cada fold como un universo independiente. Masters mismo dice: "*Personally, I prefer to focus on market patterns that are universal [...] my own habit is to permute all market changes after the first fold's training set as a single large group.*" (Masters)

Detalle clave: el oráculo SIEMPRE ve la y real

En la corrida real y en cada permutación, el oráculo recibe la y **verdadera** (capturada vía closure), nunca la permutada. Su ventaja es ver el futuro con ruido; eso no cambia entre permutaciones. Lo que sí cambia es que sus señales se evalúan contra `ret_perm` (retornos desordenados), así que en las permutaciones el oráculo "acierta el signo" pero contra el retorno equivocado. Ese diseño es lo que da al oráculo el rol de **referencia con señal verdadera** en el experimento.

5 Resultados e interpretación

Corrimos 1,000 permutaciones por modelo en cada test (in-sample y walk-forward), en paralelo con `joblib`. Toda la simulación tardó varios minutos por modelo en LGBM; en total, $ $2-3 horas de cómputo. Los plots fueron guardados como PNG en `results/` y se cargan a continuación.

5.1 Tabla resumen

```
rows = []
for name in ['oracle', 'lgbm', 'lgbm_clean', 'random']:
    rows.append({
        'Modelo'    : name,
        'PF real IS': is_results[name]['real_pf'],
        'p-value IS': is_results[name]['p_value'],
        'PF real WF': wf_results.get(name, {}).get('real_pf', np.nan),
        'p-value WF': wf_results.get(name, {}).get('p_value', np.nan),
    })
summary = pd.DataFrame(rows).set_index('Modelo')
```

Modelo	PF real IS	p-value IS	PF real WF	p-value WF
oracle	5.176	0.0010	5.390	0.0010
lgbm	16.910	0.0440	1.063	0.2817
lgbm_clean	1.877	0.0180	—	—
random	0.963	0.8292	1.018	0.3407

5.2 In-Sample MCPT — histogramas

5.2.1 Oráculo

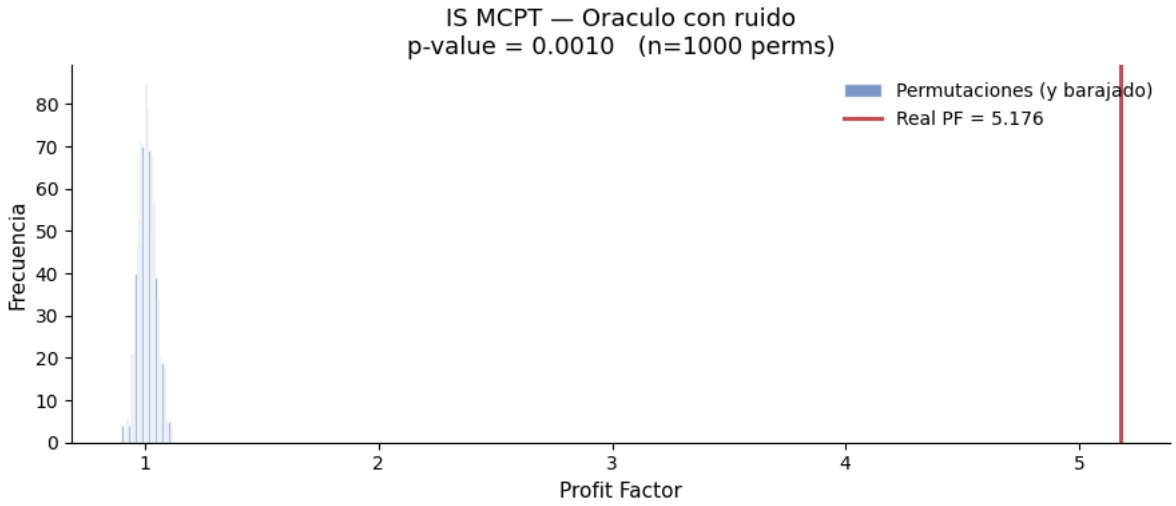

Figure 2: IS MCPT — Oráculo. La línea roja (PF real = 5.176) cae completamente a la derecha del abanico de permutaciones (centrado en 1). p-value = 0.0010, el mínimo posible con 1,000 permutaciones.

El oráculo tiene la señal **plantada por nosotros**. Bajo permutación, la señal del oráculo (que sigue viendo la y real) se evalúa contra retornos aleatoriamente desordenados, por lo que su PF colapsa a ≈ 1. El PF real de 5.18 está varias desviaciones a la derecha de todo el abanico. **El MCPT detecta correctamente la señal.**

5.2.2 LightGBM (overfit)

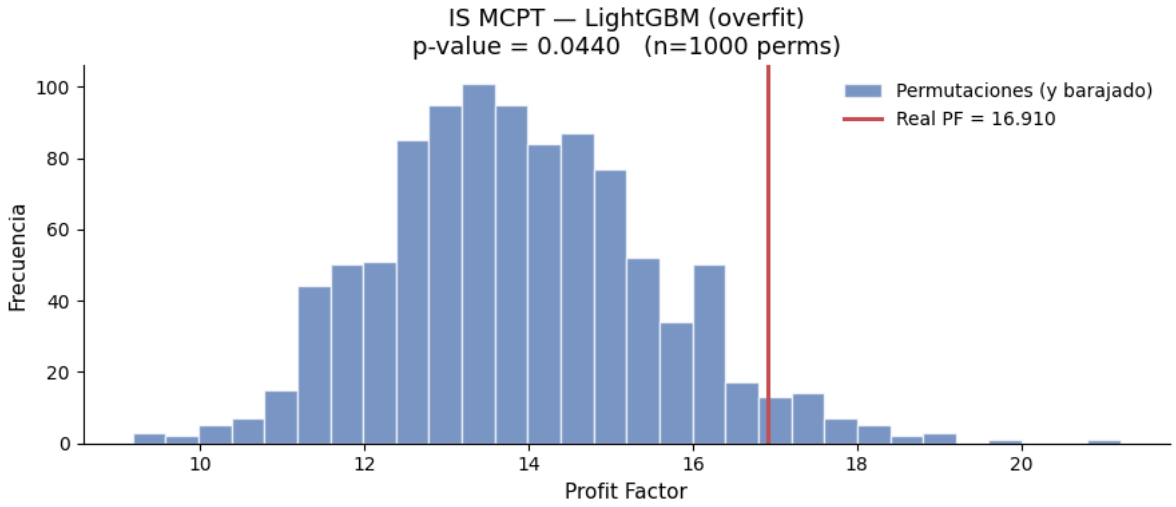

Figure 3: IS MCPT — LightGBM con 27 features. La distribución bajo H_0 está **centrada en** ≈ 14 (¡catorce!). El PF real (16.91) apenas sobresale del lado derecho. p-value = 0.044.

Este plot es el resultado más interesante de todo el reporte. **El LGBM produce un PF de 16.9 sobre datos reales, y un PF promedio de 14 sobre y aleatoriamente permutadas.** Eso es Masters en acción:

> "An overfitted trading system will perform well not only on the original data but on permuted data as well. [...] All in-sample performances, permuted and unpermuted, will be excellent."

El modelo es tan flexible que **vuelve a memorizar el ruido** cada vez que le damos un nuevo y permutado. El backtest "espectacular" (PF = 16.9) es prácticamente indistinguible de lo que se obtiene con un target aleatorio. Si un practitioner solo viera el PF real, concluiría que su sistema es genial; el MCPT muestra que casi todo ese PF es overfitting.

El p-value de 0.044 es engañosamente "significativo". La razón por la que no es 1.0 está discutida más abajo: las features están correlacionadas con la magnitud de r_{t+1}, lo que da una pequeña ventaja injusta a la corrida real.

5.2.3 LightGBM clean (sólo OHLC crudo)

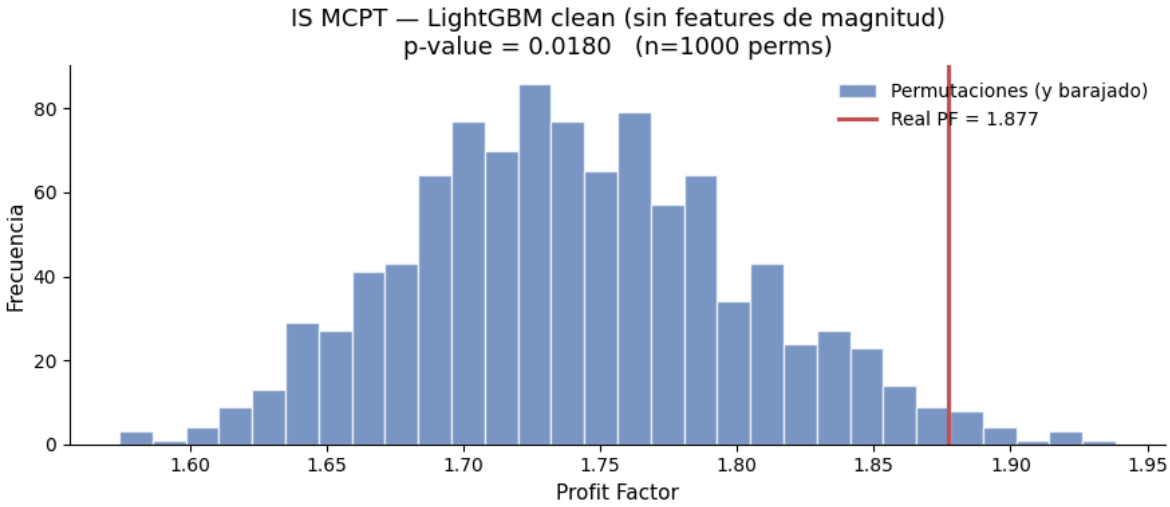

Figure 4: IS MCPT — LGBM entrenado sólo sobre `open, high, low, close` crudos. El PF real cae en PF = 1.88, en el extremo derecho del abanico de permutaciones (centrado en 1.75). p-value = 0.018.

Aquí limitamos las features a OHLC crudos para minimizar la correlación con $|r_{t+1}|$. El PF real (1.88) sigue siendo más alto que el promedio bajo H_0 (1.75), pero la **diferencia absoluta es chica**. El modelo aún encuentra "algo", pero el efecto es modesto.

La diferencia entre LGBM (PF 14 bajo H_0) y LGBM clean (PF 1.75 bajo H_0) cuantifica exactamente el sesgo inducido por features correlacionadas con la magnitud del retorno. Sin esas features, el test se comporta mucho más limpiamente.

5.2.4 Random puro

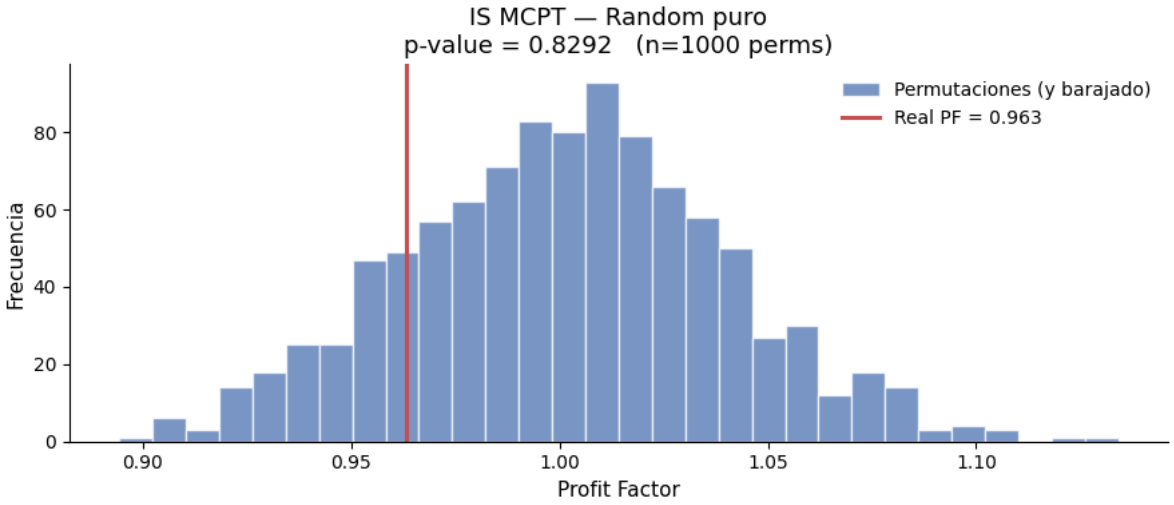

Figure 5: IS MCPT — Random. La distribución bajo H_0 está centrada en PF = 1 (como debe). El PF real (0.96) cae en el lado izquierdo. p-value = 0.83.

Control negativo: el random no tiene señal alguna. El PF real es 0.96 (ligeramente perdedor por azar) y cae cerca del centro de la distribución nula. Un p-value de 0.83 indica que no hay nada raro. **Bajo H_0 el p-value es aproximadamente uniforme**, así que cualquier valor entre 0 y 1 es esperable; obtener 0.83 es una realización particular del control negativo.

5.3 In-Sample MCPT — curvas de equity

Los histogramas muestran la distribución del estadístico final (PF). Las curvas de equity muestran **trayectorias completas** del PnL acumulado: la corrida real en rojo, las 1,000 permutaciones en gris. Visualmente:

- Si la curva roja se separa claramente del abanico gris → **hay señal**.
- Si la roja queda dentro o pegada al abanico → **no hay señal distinguible**.

5.3.1 Oráculo

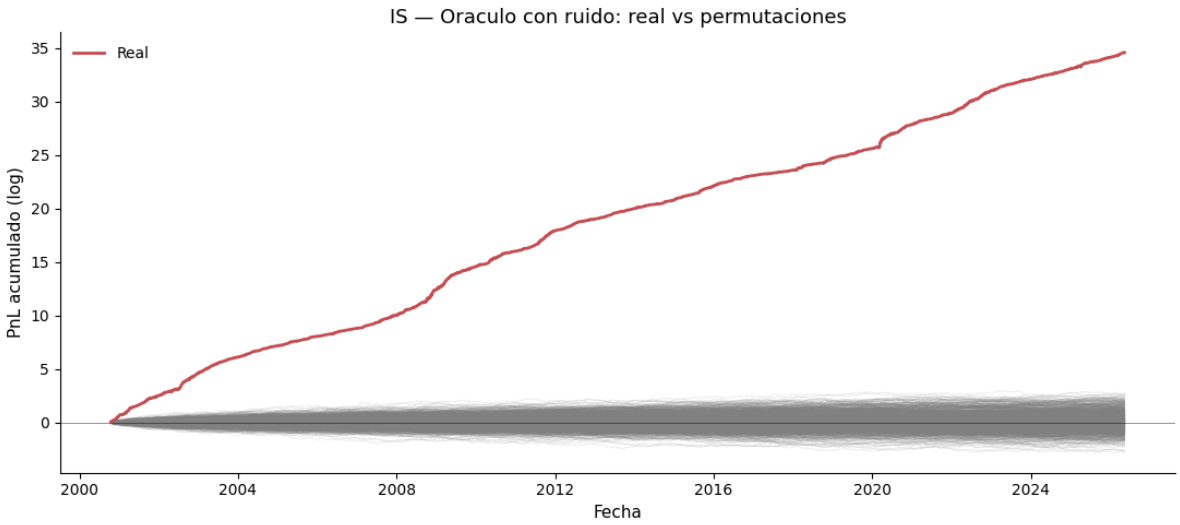

Figure 6: IS — Oráculo: la curva real (rojo, PnL acumulado de 35) está dramáticamente separada del abanico gris (todo entre ± 2). Visualmente obvio que hay señal.

5.3.2 LightGBM (overfit)

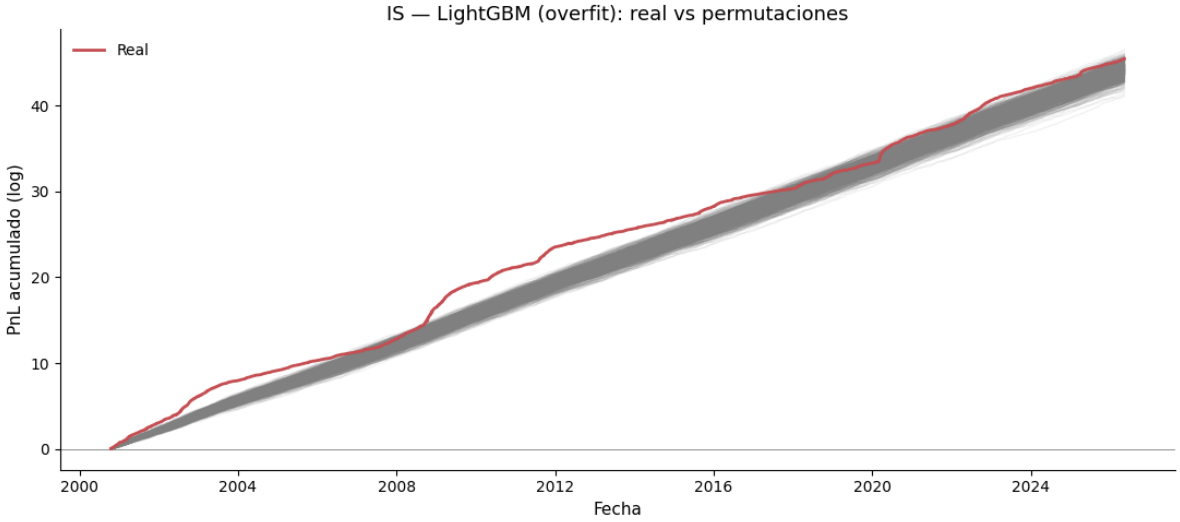

Figure 7: IS — LightGBM: la curva real (rojo) está **dentro del abanico gris**. Termina en PnL 45, pero las permutaciones también terminan en $40 - 46$. Visualmente, el modelo overfit no logra distinguirse del overfitting de las permutaciones.

Este es el mensaje pedagógico central: una curva de equity que sube limpiamente de 0 a 45 a lo largo de 26 años de "backtest" puede no significar absolutamente nada. Cualquier y aleatoriamente permutada produce algo equivalente cuando el modelo es lo suficientemente flexible.

5.3.3 LightGBM clean

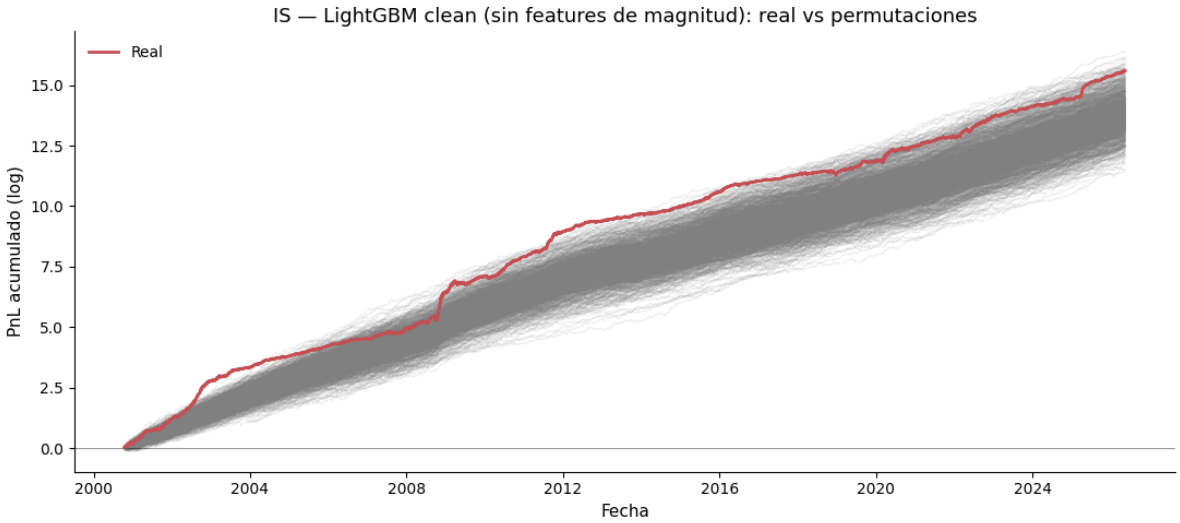

Figure 8: IS — LGBM clean: la curva real está en el borde superior del abanico, pero claramente metida adentro. Sin features de magnitud, el "edge" sobre las permutaciones es mucho menor.

5.3.4 Random

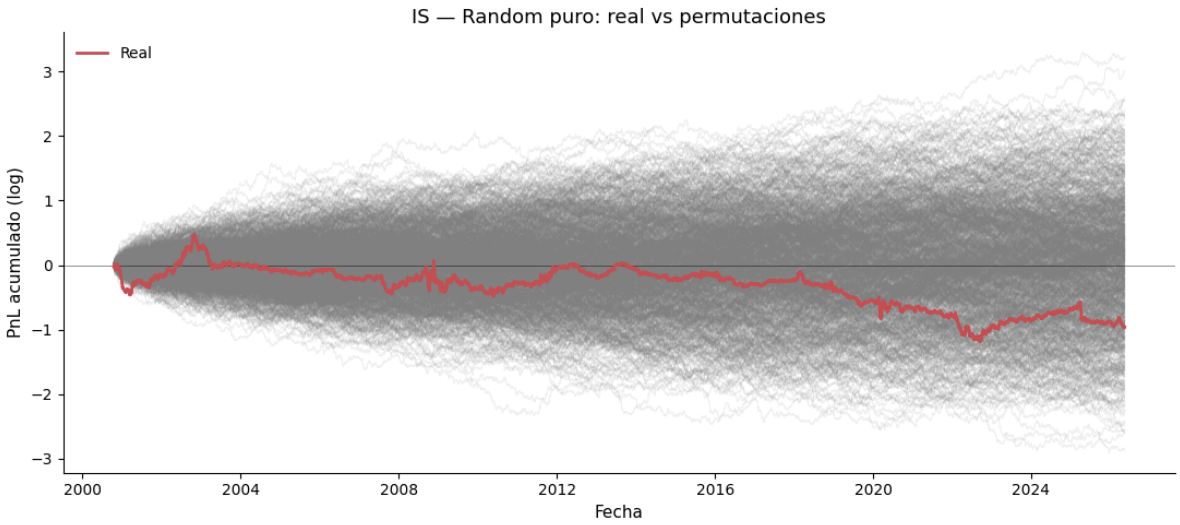

Figure 9: IS — Random: la curva real es indistinguible del abanico gris (centrada en 0).

5.4 Walk-Forward MCPT — histogramas

5.4.1 Oráculo

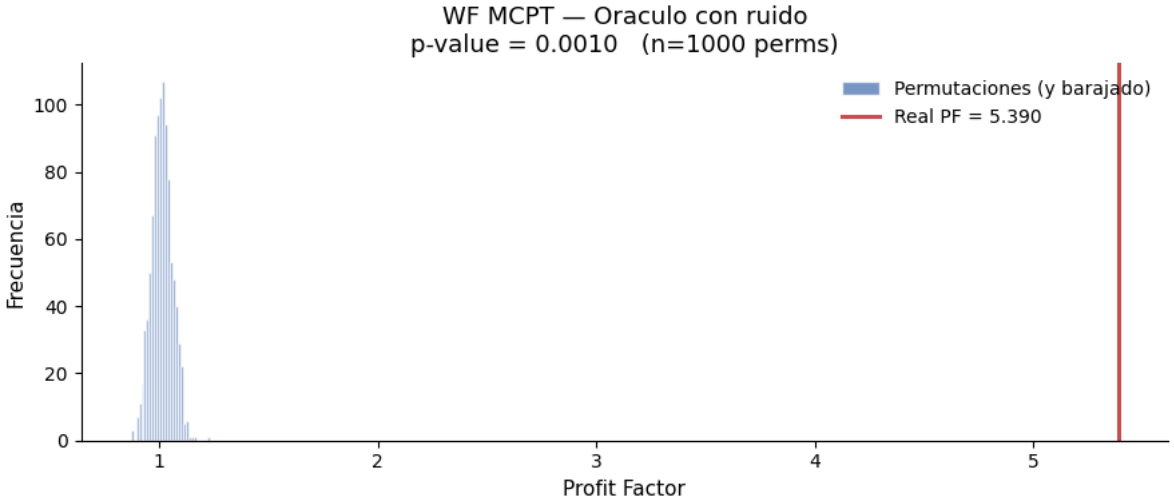

Figure 10: WF MCPT — Oráculo: idéntico al IS. PF real = 5.39, p = 0.001. La señal del oráculo es genuinamente OOS.

El oráculo, al ver la y real en cada ventana, mantiene su ventaja también OOS. p-value mínimo.

5.4.2 LightGBM (overfit) — el resultado clave del reporte

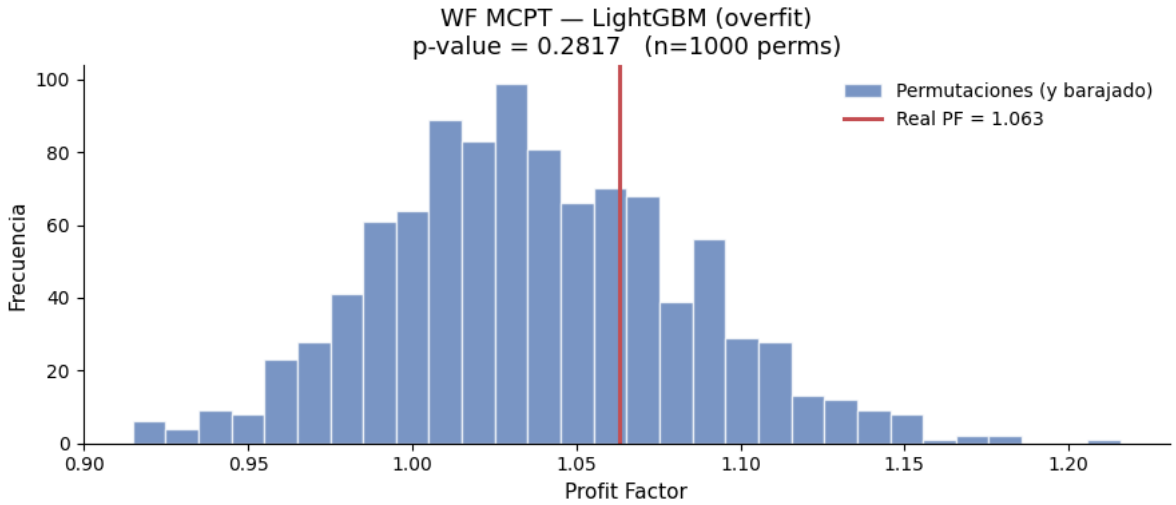

Figure 11: WF MCPT — LightGBM: PF real = 1.06, **dentro** del abanico centrado en 1.04. p-value = 0.28. **El overfit no sobrevive walk-forward.**

Este histograma, comparado con el de IS-LGBM, es la demostración empírica más limpia de overfitting que se puede pedir. **In-sample**, el LGBM producía PF de 16.9 (con permutaciones centradas en 14). **Walk-forward**, el mismo modelo, mismas features, produce PF de 1.06 con permutaciones centradas en 1.04. Toda la "magia" del backtest era memorización de ruido. Cuando el modelo tiene que predecir realmente el futuro, no hay señal que extraer.

Masters describe exactamente este patrón:

> "An overfitted trading system will perform well not only on the original data but on permuted data as well. [...] Such systems produce astonishing performance in the training period and yet produce completely random trades out-of-sample."

5.4.3 Random

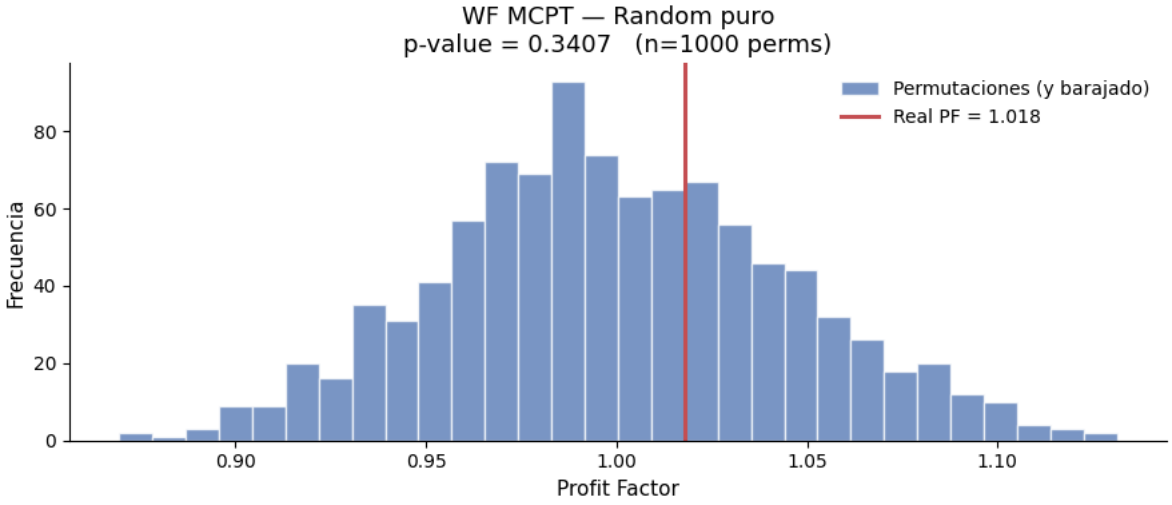

Figure 12: WF MCPT — Random: PF real = 1.02, dentro del abanico centrado en 1. p = 0.34.

Sin sorpresa. El random sigue siendo random.

5.5 Walk-Forward MCPT — curvas de equity

En estas gráficas, la línea vertical punteada marca el **inicio del rango OOS** (fin de la primera ventana de training, $\approx$ 2008). Hasta ese punto todas las trayectorias son idénticas (no se permuta). A partir de ahí el abanico se abre.

5.5.1 Oráculo

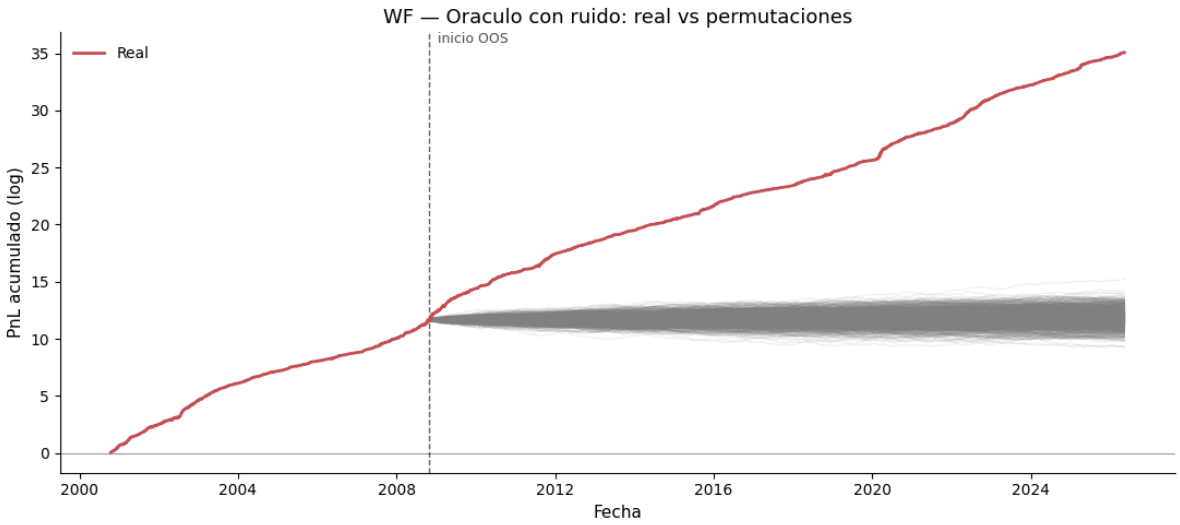

Figure 13: WF — Oráculo: hasta 2008 la curva es común. A partir de ahí la curva real (roja) sigue creciendo linealmente hasta PnL 35, mientras el abanico gris se queda pegado a 10-15. Separación dramática.

5.5.2 LightGBM

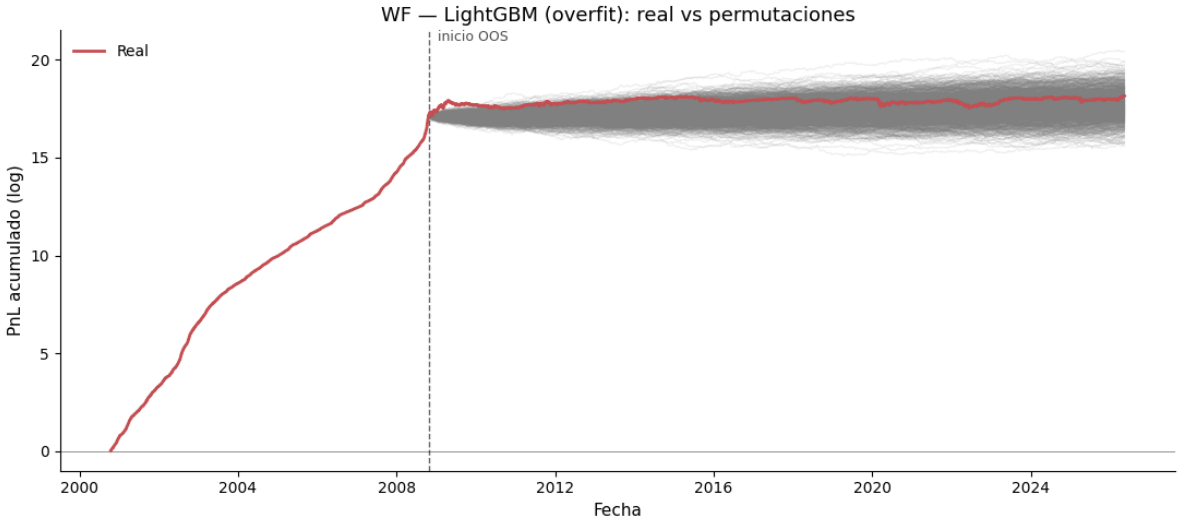

Figure 14: WF — LightGBM: hasta 2008 la curva (entrenamiento in-sample) sube de 0 a 17. A partir de 2008 (zona OOS), **la curva real se vuelve plana y queda completamente dentro del abanico**. Toda la "ganancia" pre-2008 era in-sample (training period).

Esta es probablemente la imagen más educativa del reporte. La curva pre-OOS muestra cómo se vería un "backtest" tradicional sin walk-forward: PnL subiendo limpiamente de 0 a 17 a lo largo de 8 años. Cualquier presentación de PowerPoint vendiéndote una estrategia trading con esa curva resulta convincente. **Pero a partir del momento en que el modelo tiene que predecir realmente el futuro, no hay diferencia con permutaciones aleatorias.**

5.5.3 Random

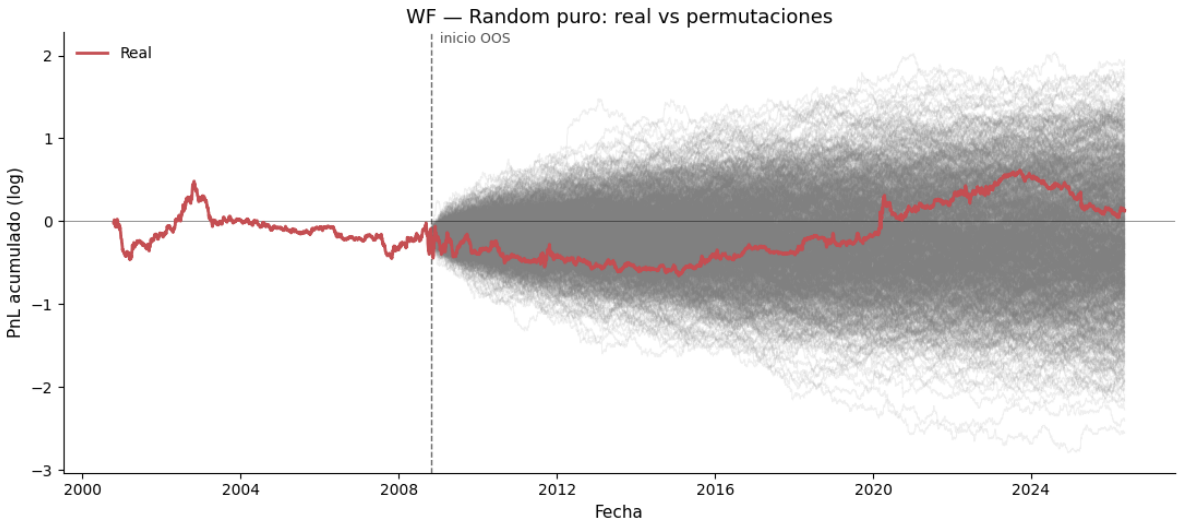

Figure 15: WF — Random: como esperado, indistinguible del abanico gris en todo el período OOS.

5.6 Discusión del sesgo IS-LGBM

El p-value IS de LGBM full (0.044) es bajo, pero no por la razón "buena" (señal genuina). Es por un sesgo que vale la pena explicar con precisión.

Fuente del sesgo: features como `rvol_5d`, `rvol_21d`, `atr_norm_14d` correlacionan con $|r_{t+1}|$ — saben cuándo el mercado está volátil, aunque no en qué dirección. Bajo Y-permutation, cuando barajamos (y, r), los retornos grandes se asocian con días donde la volatilidad reciente era cualquiera. Pero en la corrida real, los retornos grandes coinciden con días donde la volatilidad reciente era genuinamente alta — y el modelo, sin saber el signo, **igual gana** porque concentra sus posiciones (en cualquier dirección) en días donde el retorno absoluto es grande.

Este efecto rompe el supuesto de Masters de "**equal probability under H** ": las permutaciones no son intercambiables con la realidad porque la correlación entre features y $|r|$ es asimétrica.

La variante LGBM clean (sólo OHLC crudos, sin features de magnitud) elimina la mayor parte del sesgo. Que el p-value de LGBM clean (0.018) siga siendo bajo en IS sugiere que LightGBM tiene capacidad para encontrar **alguna** estructura incluso en OHLC crudo — probablemente serial dependencies de cortísimo plazo. Pero el WF demuestra que esa estructura no es explotable OOS.

6 Conclusiones

6.1 Hallazgos principales

1. **El MCPT discrimina correctamente entre modelos con señal y sin señal**. El oráculo obtiene p-values < 0.01 tanto IS como WF. El random obtiene p-values dispersos en el rango medio, consistentes con H_0.
2. **El MCPT detecta overfitting in-sample**. LightGBM produce PF in-sample de 16.9 (espectacular en un backtest tradicional), pero el promedio bajo permutación es 14: el modelo aprende ruido en cualquier y que se le presente. El WF lo confirma: el mismo modelo cae a PF = 1.06 OOS, dentro de la distribución nula. **Sin permutation testing, esto sería invisible.**
3. **Las features correlacionadas con la magnitud del retorno contaminan el Y-permutation test**. Comparar LGBM (PF nulo ≈ 14) contra LGBM clean (PF nulo ≈ 1.75) cuantifica el sesgo. La consecuencia práctica: en aplicaciones reales, **la elección de features debe considerar no sólo su poder predictivo sino su correlación con** $|r|$ para no romper la validez del MCPT.
4. **Walk-forward es el filtro definitivo**. Cualquier diferencia entre LGBM IS y LGBM WF es overfitting puro. La caída de PF 16.9 $\rightarrow$ 1.06 entre los dos tests es la firma cuantitativa del sobreajuste.

6.2 Respuesta a las preguntas iniciales

Pregunta	Respuesta
¿El MCPT IS detecta overfitting?	Sí, pero requiere mirar la *distribución bajo* H_0, no sólo el p-value. La distribución del LGBM bajo H_0 está centrada en 14, no en 1: eso es la señal de overfitting.
¿El MCPT WF detecta sistemas genuinos?	Sí. Oracle WF tiene $p < 0.001$; LGBM WF tiene $p = 0.28$. La separación es nítida.
¿Bajo H_0 el p-value es uniforme?	Aproximadamente. Random IS = 0.83, WF = 0.34. Una sola realización; para validarlo formalmente habría que repetir el experimento con muchas semillas.
¿Las features de magnitud sesgan el test?	Sí. El sesgo es cuantificable y se elimina restringiendo features.

6.3 Limitaciones

- **Una sola realización del control negativo (random)**. Para validar formalmente que el p-value es uniforme bajo H_0, habría que repetir el experimento con varios `REAL_SEED`. Computacionalmente costoso.
- **Y-permutation supone target sin autocorrelación**. El signo del log-return diario tiene autocorrelación ≈ 0 pero no exactamente. En timeframes intradía esto podría romperse.
- **No estudiamos selection bias** (Masters). Si hubiéramos probado muchos modelos y reportado sólo el mejor, los p-values de arriba serían optimistas. Lo evitamos al fijar los tres modelos *a priori*.
- **Single asset**. Generalizar a otros activos / mercados requiere repetir.

6.4 Trabajos futuros

- Repetir con permutación de precios al estilo Masters para sistemas de trading puros, en lugar de Y-permutation para predictivos.
- Bootstrap test (Masters) para comparar p-values bajo distintas formulaciones.
- Extender al algoritmo con corrección por selection bias (Masters) para evaluar familias enteras de modelos.
- Permutación por fold (en lugar de global) para datos con regímenes claramente cambiantes.

7 Referencias

- Masters, T. (2018). *Testing and Tuning Market Trading Systems: Algorithms in C++*. Apress. Capítulo 7: "Permutation Tests".
- Aronson, D. (2007). *Evidence-Based Technical Analysis*. Wiley.
- Ke, G. et al. (2017). "LightGBM: A Highly Efficient Gradient Boosting Decision Tree". *NeurIPS*.
- Yahoo Finance: datos OHLCV de SPY, vía la librería `yfinance`.

8 Anexo de código

Todo el código corre en Python 3.11 con `conda run -n rappi`. Las dependencias relevantes:

```
numpy        >= 1.26
pandas       >= 2.0
yfinance     >= 0.2
lightgbm     >= 4.0
```

```
matplotlib   >= 3.8
seaborn      >= 0.13
joblib       >= 1.3
tqdm         >= 4.66
```

8.1 Configuración

```python
import warnings; warnings.filterwarnings('ignore')
import numpy as np, pandas as pd, yfinance as yf
import matplotlib.pyplot as plt, seaborn as sns
from lightgbm import LGBMClassifier
from tqdm.auto import tqdm
from joblib import Parallel, delayed

RNG_SEED     = 67
N_PERM       = 1000
TRAIN_WINDOW = 252 * 8       # 8 años
WF_STEP      = 252 * 3       # 3 años (6 folds OOS)
ORACLE_NOISE = 1.0
N_JOBS       = 8
REAL_SEED    = 7

np.random.seed(RNG_SEED)
```

8.2 Descarga de datos

```python
spy = yf.download('SPY', start='2000-01-01', auto_adjust=True, progress=False)
if isinstance(spy.columns, pd.MultiIndex):
    spy.columns = spy.columns.droplevel(1)
spy.columns = [c.capitalize() for c in spy.columns]
```

8.3 Función de Profit Factor

```python
def profit_factor(signal: pd.Series, log_ret: pd.Series) -> float:
    pnl = signal.values * log_ret.values
    pnl = pnl[~np.isnan(pnl)]
    wins = pnl[pnl > 0].sum()
```

```
    loss = np.abs(pnl[pnl < 0].sum())
    return wins / loss if loss > 1e-10 else float('inf')
```

8.4 In-sample MCPT (completo)

```
def _permute_pair(y, ret, seed):
    rng = np.random.default_rng(seed)
    idx = rng.permutation(len(y))
    return (pd.Series(y.values[idx],   index=y.index),
            pd.Series(ret.values[idx], index=ret.index))

def run_is_mcpt(model_name, X, y, log_ret, n_perm=N_PERM,
                n_jobs=N_JOBS, real_seed=REAL_SEED):
    def _signal(y_for_training, seed):
        if model_name == 'oracle':
            return predict_oracle(y, seed=seed)
        elif model_name in ('lgbm', 'lgbm_clean'):
            return predict_lgbm(train_lgbm(X, y_for_training), X)
        elif model_name == 'random':
            return predict_random(X, seed=seed)

    def _one_perm(seed):
        y_perm, ret_perm = _permute_pair(y, log_ret, seed)
        sig = _signal(y_perm, seed)
        return profit_factor(sig, ret_perm)

    real_pf = profit_factor(_signal(y, real_seed), log_ret)

    perm_pfs = Parallel(n_jobs=n_jobs, backend='loky')(
        delayed(_one_perm)(s) for s in range(1, n_perm + 1)
    )
    perm_pfs = np.array([pf for pf in perm_pfs if np.isfinite(pf)])
    k     = int((perm_pfs >= real_pf).sum())
    p_val = (k + 1) / (len(perm_pfs) + 1)
    return {'real_pf': real_pf, 'perm_pfs': perm_pfs, 'p_value': p_val}
```

8.5 Walk-Forward MCPT (completo)

```
def _wf_signal_for_window(model_name, X_tr, y_tr_for_training,
                          y_real_window, X_win, seed):
    if model_name == 'oracle':
        return predict_oracle(y_real_window, seed=seed)     # ve y real
    elif model_name == 'lgbm':
        return predict_lgbm(train_lgbm(X_tr, y_tr_for_training), X_win)
    elif model_name == 'random':
        return predict_random(X_win, seed=seed)

def _wf_one(model_name, X, y, log_ret, y_used, ret_used, real_seed=REAL_SEED):
    n = len(X); signals = []; rets_eval = []

    # Train inicial (común a todas las permutaciones)
    X_init       = X.iloc[:TRAIN_WINDOW]
    y_init_real  = y.iloc[:TRAIN_WINDOW]
    y_init_train = y_used.iloc[:TRAIN_WINDOW]
    ret_init     = ret_used.iloc[:TRAIN_WINDOW]
    signals.append(_wf_signal_for_window(
        model_name, X_init, y_init_train, y_init_real, X_init, real_seed))
    rets_eval.append(ret_init)

    # Folds OOS
    for start in range(TRAIN_WINDOW, n, WF_STEP):
        end       = min(start + WF_STEP, n)
        X_tr      = X.iloc[start - TRAIN_WINDOW : start]
        y_tr      = y_used.iloc[start - TRAIN_WINDOW : start]
        X_te      = X.iloc[start : end]
        y_te_real = y.iloc[start : end]
        r_te      = ret_used.iloc[start : end]
        sig = _wf_signal_for_window(
            model_name, X_tr, y_tr, y_te_real, X_te, real_seed + start)
        signals.append(sig); rets_eval.append(r_te)

    return pd.concat(signals), pd.concat(rets_eval)

def _wf_one_perm(seed, model_name, X, y, log_ret):
    y_used   = y.copy()
    ret_used = log_ret.copy()
    rng      = np.random.default_rng(seed)
    idx      = rng.permutation(len(y) - TRAIN_WINDOW)
```

```
    y_used.iloc[TRAIN_WINDOW:]   = y.iloc[TRAIN_WINDOW:].values[idx]
    ret_used.iloc[TRAIN_WINDOW:] = log_ret.iloc[TRAIN_WINDOW:].values[idx]

    sig, ret = _wf_one(model_name, X, y, log_ret, y_used, ret_used)
    sig_oos = sig.iloc[TRAIN_WINDOW:]
    ret_oos = ret.iloc[TRAIN_WINDOW:]
    valid   = ret_oos.notna()
    return profit_factor(sig_oos[valid], ret_oos[valid])

def run_wf_mcpt(model_name, X, y, log_ret, n_perm=N_PERM,
                n_jobs=N_JOBS, real_seed=REAL_SEED):
    sig_real, ret_real = _wf_one(model_name, X, y, log_ret, y, log_ret, real_seed)
    sig_oos = sig_real.iloc[TRAIN_WINDOW:]
    ret_oos = ret_real.iloc[TRAIN_WINDOW:]
    valid   = ret_oos.notna()
    real_pf = profit_factor(sig_oos[valid], ret_oos[valid])

    perm_pfs = Parallel(n_jobs=n_jobs, backend='loky')(
        delayed(_wf_one_perm)(s, model_name, X, y, log_ret)
        for s in range(1, n_perm + 1)
    )
    perm_pfs = np.array([pf for pf in perm_pfs if np.isfinite(pf)])
    k     = int((perm_pfs >= real_pf).sum())
    p_val = (k + 1) / (len(perm_pfs) + 1)
    return {'real_pf': real_pf, 'perm_pfs': perm_pfs, 'p_value': p_val}
```

8.6 Notebook completo

El notebook con todo el código ejecutable está en `mcpt_calibration.ipynb`. La simulación toma aproximadamente 2-3 horas en una máquina con 8 cores. Para reproducir resultados exactos:

```
conda run -n rappi jupyter nbconvert --to notebook --execute mcpt_calibration.ipynb
```

Las semillas (`RNG_SEED=67`, `REAL_SEED=7`) garantizan reproducibilidad bit-exact de la corrida real. Las permutaciones usan semillas $1, 2, \ldots, m$ vía `np.random.default_rng(seed)`, así que también son reproducibles.